1. JUSTIFICACIÓN

En las prácticas anteriores, de una forma u otra, se han presentado, usado, todos los elementos que forman parte de un sistema de domótica (Figura 1), es decir, hemos usado la plataforma Arduino UNO como controlador, pulsadores como ejemplo de sensores digitales y relés/triacs como ejemplo de actuadores. También, hemos usado una aplicación Java como ejemplo de interfaz gráfico y hemos comunicado mediante un puerto serie el interfaz gráfico con el controlador, con lo que hemos dotado a nuestro sistema de prácticas de la capacidad de intercambiar información entre los controladores y los interfaces gráficos.

Las comunicaciones implementadas hasta ahora han sido muy básicas y de una forma u otra, para la transferencia de información hemos usado la lógica, no habiéndose tenido en cuenta los problemas que pueda surgir al no disponer de un protocolo de comunicaciones normalizado. Así pues, para dotar a nuestro sistema domótico de una mayor flexibilidad, la identificación y selección, tanto de los dispositivos, como de los elementos que los integran, sobre la hasta ahora línea serie usada para transferir datos montaremos un protocolo de comunicaciones.

Así pues, como principal objetivo de la práctica, no proponemos adoptar un protocolo de comunicaciones que:

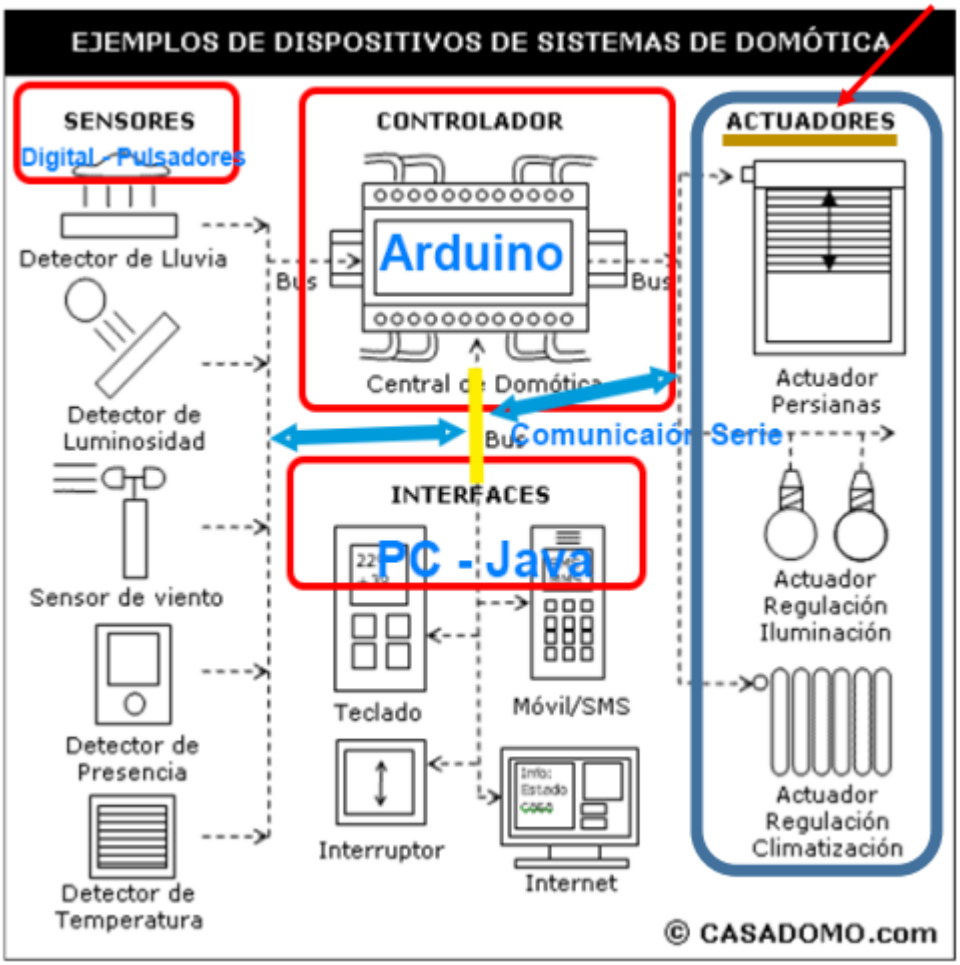

Figura 1.- Esquema sistema de automatización domiciliaria

- Tenga suficiente implantación en el mundo real
- Sea fácil de implementar
- Sea fácilmente adaptable a cualquier situación que pueda surgir,
- Y, sobre todo, que sea público y gratuito.

Con estas premisas, el protocolo elegido es **ModBus**.

2. FUNDAMENTO TEÓRICO

ModBus, originalmente fue pensado para conectar dispositivos con una configuración *maestro-esclavo*, en la que todas las comunicaciones del sistema están supervisadas por el maestro. Implementaciones posteriores, permiten que las comunicaciones puedan ser inicias por cualquier dispositivo conectado a la red.

Dentro de la red ModBus, cada dispositivo es identificado mediante una dirección que debe ser única para toda la red, la cual es introducida en la trama para indicar el destino del mensaje. Cada mensaje es recibido por todos los dispositivos conectados, aunque sólo es ejecutado por el dispositivo al que va destinado. Aquel cuya dirección coincida con la dirección de esclavo incluida en el mensaje. Algunos mensajes especiales,

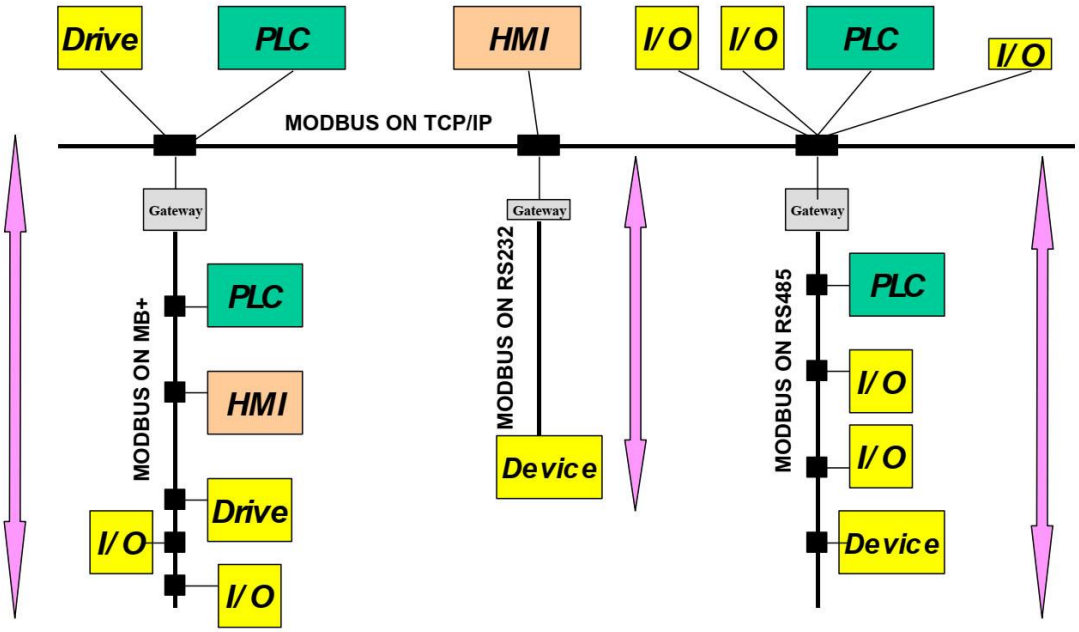

Figura 2.- Ejemplo de arquitectura de red MODBUS.

http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf

como los de tipo "**broadcast**", que están destinados a todos los dispositivos de la red a la vez.

Este protocolo, originalmente, fue diseñado para realizar comunicaciones en serie entre dispositivos, aunque el éxito alcanzado, por su versatilidad y capacidad de adaptación a distintos entornos hizo que fuese ampliado a otros estándares de comunicación, y así poder adaptarlo para comunicación en redes TCP/IP, UDP, así como la adaptación del estándar con distintas configuraciones de tramas (Figura 2).

Las especificaciones para la implementación del protocolo ModBus las podemos encontrar aquí. Donde podréis encontrar las líneas maestras para su implementación, especialmente, y puesto que en el desarrollo de nuestras prácticas, usaremos como medio para las comunicaciones líneas series, para su implementación a través de una línea serie.

Para implementaciones de ModBus en líneas serie, usamos, principalmente dos estándares, el modelo RTU, donde las tramas (PDU – Unidad de Datos del Protocolo) están separadas por un silencio de 3.5 caracteres al comienzo y final de la trama, como muestra la Figura 3.

Figura 3.- Trama ModBus RTU

El segundo estándar usado para la implantación serie del protocolo es el estándar ASCII, cuya trama se muestra en la Figura 4. Aquí, las tramas están mejor definidas, desde el punto de vista que todas comienzan y finalizan de la misma manera, es decir, comienzan con el carácter (:) y finalizan con un retorno de carro (CR) y un inicio de línea (LF). Pero, si en un principio, esto pudiera facilitar la implementación del protocolo, la desventaja que tiene este estándar es que todos los datos son transferidos en código ASCII, es decir, mientras que para el estándar RTU los datos son enviados en binario, y para el valor 3 envía un byte 0x03, el estándar ASCII envía cada uno de los caracteres en ASCII, es decir, envía "0" y "3", o lo que es lo mismo, 0x30 y 0x33, por lo que nos hemos decantado para usar en nuestras prácticas por el estándar RTU.

Figura 4.- Trama ModBus ASCII

3. TRAMA MODBUS

Como podemos ver en la Figura 3, la trama RTU ModBus tiene cuatro partes bien diferenciadas:

- Dirección
- Función
- Datos
- Ckecksum

3.1 Datos

ModBus, maneja principalmente cuatro tipos de datos: **Entradas Digitales** (*Discrete Input*), **Salidas Digitales** (*Coils*) para manejar datos booleanos, de un solo bits, y **Registros de Entrada** (*Input Register*) y **Registros de retención** (*Holding Register*) para manejar datos "analógicos" palabras con una longitud de 2 bytes, 16 bits.

Generalmente, estos datos, son manejados en cuatro bloques, o rangos de dirección, separados, almacenados en el dispositivo que los genera/maneja, el esclavo. El dispositivo maestro podrá acceder a estos datos para recibir información, así como enviar/configurar elementos del dispositivo esclavo. En este sentido, en función de la operación que se vaya a realizar en cada uno de estos bloques de memoria/datos se definen unos permisos de acceso (Tabla 1).

Tabla 1.- Modelo de datos de ModBus

Bloque de Memoria	Tipos de Datos	Acceso de Maestro	Acceso de Esclavo
Salidas Digitales	Booleano	Lectura/Escritura	Lectura/Escritura
Entradas Digitales	Booleano	Sólo Lectura	Lectura/Escritura
Registros de Retención	Palabra Sin Signo	Lectura/Escritura	Lectura/Escritura
Registros de Entrada	Palabra Sin Signo	Sólo Lectura	Lectura/Escritura

Estos bloques de datos, son conceptuales, y aunque habitualmente, son manejados separadamente, no hay ningún problema para que puedan ser manejaos de forma conjunta, e incluso solaparse unos con otros. Es la versatilidad que nos proporciona ModBus. La definición de las direcciones es responsabilidad del dispositivo esclavo, así como la descripción y manejo de los bloques. Todo ello es una parte importante en la descripción y funcionalidad del dispositivo esclavo.

Cada registro dentro de un bloque de memoria es seleccionado mediante una dirección de 16 bits, y está enumerado empezando por el elemento 1 hasta un valor máximo de 65536 elementos (2^{16} → 16 bits). No tienen por qué estar disponible todos los registros del bloque. Es potestad del esclavo que elementos definir, en función de sus necesidades de operación.

3.2 Dirección del dispositivo

La palabra de dirección de la trama ModBus tiene una longitud de un byte, lo que nos permite acceder de forma individualizada hasta 247 dispositivos esclavos. Cada dispositivo esclavo, tiene asignada una única dirección entre 1 y 247, quedando las direcciones 0 y 248-255 reservadas para direccionamientos especiales, como el broadcast.

3.3 Función

Con este campo, la trama ModBus (Figura 3), indica lo operación a realizar con el dispositivo destinatario del mensaje. Este campo tiene una longitud de 1 byte, y permite diferenciar entre 127 operaciones diferentes. El bit más significativo del código de operación, es reservado para mostrar errores en el proceso de ejecución de la operación.

ModBus tiene muchas operaciones que pueden ser consideradas estándar, que son reconocidas por la mayoría de los usuarios y fabricantes (Tabla 2), aunque otras muchas funciones pueden ser definidas en el esclavo por el fabricante, no incluidas en el estándar.

Tabla 2.- funciones ModBus más comunes

Código	Descripción
01 (0x01 Hex)	Leer señales discretas de salida
02 (0x02 Hex)	Leer señales discretas de entrada
03 (0x03 Hex)	Leer registros de retención (Salidas)
04 (0x04 Hex)	Leer registros de entrada
05 (0x05 Hex)	Escritura de una señal discreta de salida
06 (0x06 Hex)	Escritura de un solo registro de retención (Salida)
08 (0x07 Hex)	Reset del Esclavo
15 (0x0F Hex)	Escritura de múltiples salidas discretas
16 (0x10 Hex)	Escritura de múltiples registros de retención)

La comunicación maestro-esclavo es muy simple (Figura 5), el maestro envía un requerimiento/comando al esclavo, y este responde con una trama en la que incluye la respuesta, en la que, devolverá los datos solicitados por el maestro, y en su defecto, si se ha producido un error indicará la naturaleza del mismo.

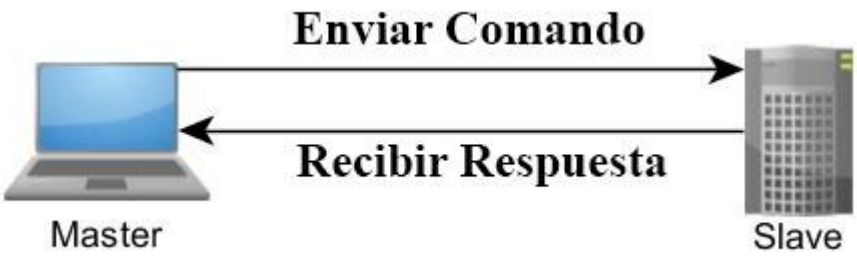

Figura 5. Relación Maestro-Esclavo.

https://www.ni.com/es-es/innovations/white-papers/14/the-modbus-protocol-in-depth.html

Las tramas con las que el maestro envía un comando (Figura 6) y la respuesta enviada por el esclavo, tiene siempre una estructura bien definida, como se muestra en la Figura 6 y Figura 7.

Pregunta : Maestro → Esclavo						
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)
		Dirección Comienzo		Número de registros		
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo	

Figura 6.- Trama Maestro - Esclavo

Respuesta : Esclavo → Maestro					
Dirección (1 byte)	Función (1 byte)	Datos			Ctrl. Err. (CRC)
		Nº Bytes de respuesta	Bytes Respuesta		
		1 Byte	1 Byte …	… 1 Byte	

Figura 7.- Trama Esclavo Maestro

En la Figura 8, se muestra un ejemplo de trama enviada por el maestro al esclavo, catalogado con la dirección 0x0A, con la función 0x01 (Leer señales discretas de salida), la dirección del esclavo donde se encontrarán estas salidas será la 0x4A1 y a partir de ahí, se leerá un solo registro. Sin embargo, cuando en el requerimiento enviado por el maestro, el esclavo no tiene respuesta y se produce un error, este, se indica en la trama de respuesta. Para la trama ejemplo de la Figura 8, el esclavo no encuentra respuesta, y responde con una trama de error, mostrada en la Figura 9.

Pregunta : Maestro → Esclavo						
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)
		Dirección Comienzo		Número de registros		
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo	
0x0A	0x01	0x04	0xA1	0x00	0x01	0x4F

Figura 8.- Ejemplo de trama enviada por el maestro

Respuesta Error : Esclavo → Maestro			
Dirección (1 byte)	Función (1 byte)	Código Error (1 Byte)	Ctrl. Err. (CRC)
0x0A	0x81	02	0x73

Figura 9.- Trama ejemplo de error Esclavo - Maestro

En el estándar ModBus, podemos encontrar algunos códigos de error, que la mayoría de usuarios y fabricantes entienden, y que son mostrados en la Figura 10.

Código	Tipo de Error	Significado
01	Función Ilegal	La función recibida no está permitida en el Esclavo
02	Dirección Ilegal	La dirección está fuera del rango permitido
03	Dato Ilegal	El dato contiene un valor no válido
04	Fallo en el Dispositivo	El controlador no responde o ha ocurrido un error
05	Reconocimiento (ACK)	Se ha aceptado la función y se ha procesado
06	Dispositivo Ocupado	El mensaje ha sido recibido sin error, pero el dispositivo no puede procesarlo en este momento
07	Reconocimiento Negativo (NAK)	La función solicitada no puede realizarse en este momento

Figura 10.- Códigos de error

Sin embargo, cuando el esclavo interpreta correctamente la instrucción recibida del maestro, la ejecuta y procede en consecuencia enviando la respuesta como se muestra en la Figura 11 y la Figura 12.

Pregunta : Maestro → Esclavo							
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)	
		Dirección Comienzo		Número de registros			
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo		
01_H	01_H	00_H	01_H	00_H	01_H	crc-L	Crc-H

Figura 11.- Ejemplo de requerimiento Maestro

Respuesta : Esclavo → Maestro					
Dirección Esclavo	Función	Datos		Ctrl. Err. (CRC)	
		Nº Bytes de respuesta	Bytes Respuesta		
01_H	01_H	01_H	01_H	crc-L	crc-H

Figura 12.- Respuesta del esclavo al requerimiento anterior

Si nos fijamos en la Figura 12, vemos que el esclavo responde indicando su propia dirección, la función a la que responde (Incluida en la trama del maestro), y los datos incluidos en la respuesta, comenzando por el número de bytes que incluye esta respuesta, seguido por los bytes incluidos en la misma. Para el conjunto requerimiento-Respuesta de la Figura 11 y la Figura 12, se responde a la petición del maestro en la que solicita el estado de una señal de salida discreta (01_H) con un byte (01_H) de valor (01_H).

En el caso de la lectura de entradas digitales, el comando sería 02_H. En la Figura 13 se muestra la trama de requerimiento de cuatro registros digitales de entrada. La Figura 14 muestra la respuesta del esclavo con el estado de cada uno de los cuatro registros.

Pregunta : Maestro → Esclavo							
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)	
		Dirección Comienzo		Número de registros			
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo		
01_H	02_H	00_H	01_H	00_H	04_H	crc-L	Crc-H

Figura 13.- Ejemplo de requerimiento Maestro para entradas digitales

Respuesta : Esclavo → Maestro					
Dirección Esclavo	Función	Datos		Ctrl. Err. (CRC)	
		Nº Bytes de respuesta	Bytes Respuesta		
01_H	02_H	01_H	$0F_H$	crc-L	crc-H

Figura 14.- Respuesta del esclavo al requerimiento anterior

Cuando se trata de dispositivos digitales, cada byte tiene capacidad para incluir información de ocho dispositivos distintos, uno por cada bit. La Figura 15 muestra cómo interpretar el contenido del byte de respuesta en la Figura 14 donde se incluye información acerca de cuatro dispositivos.

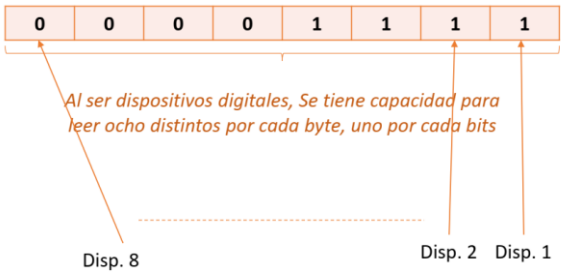

Figura 15.- significado del byte leído para dispositivos discretos.

Para leer registros de retención, valores analógicos *o aquellos cuyo contenido no se pueda expresar sólo con un bit*, podemos plantear la instrucción de la Figura 16, donde el maestro solicita al esclavo el contenido de los registros 0x54 y 0x55

Pregunta : Maestro → Esclavo							
Dirección (1 byte)	Función (1 byte)	Datos				Ctrl. Err. (CRC)	
		Dirección Comienzo		Número de registros			
		Byte Alto	Byte Bajo	Byte Alto	Byte Bajo		
01_H	03_H	00_H	54_H	00_H	02_H	crc-L	Crc-H

Figura 16.- Trama ModBus para la lectura de registros de retención

La Figura 17 muestra la trama de respuesta para la lectura de los registros de retención.

Respuesta : Esclavo → Maestro								
Dirección Esclavo	Función	Longitud Datos	Datos				Ctrl. Err. (CRC)	
			H(0x54)	L(0x54)	H(0x55)	L(0x55)		
01_H	03_H	04_H	FF_H	FF_H	FF_H	FF_H	crc-L	crc-H

Figura 17.- Trama de respuesta para la lectura de registros de retención.

4. MATERIAL

Se proporciona al alumno:

- Un proyecto Eclipse Java completamente funcional con un JButton que al ser pulsado envía una trama ModBus por el puerto serie con el comando (05_H), escritura señal discreta de salida. Este proyecto se completa con las librerías java necesarias para que funcione correctamente. En concreto, myModBus y CommTransport, como las encargadas de gestionar las comunicaciones serie para generar e interpretar las tramas ModBus

- Un proyecto Eclipse Arduino completamente funcional que recibe la trama enviada por la aplicación Java y activa el RELE de DomoBoard. Incluye una clase (ModbusSlave) que gestiona las comunicaciones serie para generar e interpretar las tramas ModBus.

4.1 myModBus

Esta librería, dispone de una función, `InitComunications`, que recibe los datos necesarios para generar una trama ModBus como se muestra en la Figura

```
String val = "1";          //Valor a escribir en el registro --> ON
String reg = Integer.toString(MB_Registers.MB_RELE);      //Registro ModBus a escribir
String slaveAdd = "1";
String[] args = {slaveAdd, String.valueOf(Const_Modbus.WRITE_COIL), reg, val};

//Iniciamos Comunicación
Modbus.InitComunication(args, sn_Transport);
```

Figura 18.- Gestión envío trama ModBus

4.2 ModBusSlave

Esta clase usada en arduino se encarga de leer el puerto serie e interpretar las tramas ModBus que lleguen y envía a través del puerto serie las tramas generadas en el controlador Arduino. Para que funcione correctamente, antes de ser usada, esta clase debe ser configurada como se muestra en la Figura 19.

```
void Init_RTU_Modbus()
{
      /* configure modbus communication
       * 19200 bps, 8E1, two-device network */
      /* Second step MBS: configure */
      /* the Modbus slave configuration parameters */
      const unsigned char       SLAVE  = ADDR_SLAVE;            //Address SLAVE
      const long                BAUD   = SERIAL_BPS;
      const char                PARITY       = SERIAL_PARITY;
      const char                TXENPIN = 0;  //EN_485;


      //Para la conexión 485/ModBus usamos
      Serial485 = &Serial;

      //We configure the ModBus Register Banks
      mbs.set_BankCoils(Cregs, MB_O_COILS);

      mbs.configure(SLAVE,BAUD,PARITY,TXENPIN);

      //Sets up event that occurred when Discrete register are updated
      mbs.writecoil = writecoil;
      }
```

Figura 19.- función para configurar la clase ModBusSlave

5. MONTAJE

El alumno, para completar la práctica debe conocer y poder realizar modificaciones en las aplicaciones proporcionadas según se indican en los apartados siguiente.

I. Estudiar y reconocer los proyectos propuestos e interpretar en ellos el procedimiento para enviar y reconocer tramas ModBus.

II. Modificar la aplicación Java con un segundo JButton que envíe una trama ModBus para activar un segundo registro de salida discreta, el TRIAC de la placa DomoBoard.

III. Modificar la aplicación Arduino para recibir la trama anterior y que active el TRIAC de DomoBoard.

a.- También, añadir en la aplicación java dos botones, uno para apagar el RELÉ y otro para apagar el TRIAC. Modificar, si es necesario, la aplicación C para que atienda estos requerimientos.

IV. Modificar la aplicación arduino para que los pulsadores (Btn.1 y Btn.2) funcionen como interruptores, del mismo modo a como lo hacían en la práctica 1. Los interruptores

implementados ahora deben ser coherentes con el protocolo ModBus, es decir, si el RELÉ/TRIAC (R/T) ha sido activado mediante una trama ModBus enviada desde la aplicación JAVA, al pulsar el botón correspondiente, el R/T será desactivado. Sucesivas pulsaciones de los botones, junto con envíos desde la aplicación JAVA activará, desactivará, según corresponda el R/T. Con todo esto, queremos decir que los procedimientos usados por el interruptor programado en Arduino para el accionamiento del R/T deben ser los mismos que los usados por la trama ModBus enviada desde la aplicación recibida